

```

import pygame
import random
import math

# Initialize Pygame
pygame.init()
pygame.font.init()

# Screen settings (tight around maze + text)
WIDTH, HEIGHT = 700, 300
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Robot Maze Decision-Making Simulation")

# Colors
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
BLUE = (0, 0, 255)
GREEN = (0, 255, 0)

# Font
font = pygame.font.SysFont('Arial', 20)

def draw_text(surface, text, pos, color=BLACK):
    img = font.render(text, True, color)
    surface.blit(img, pos)

# Robot settings
robot_pos = [30, 200] # Start at entrance on left (center vertically)
robot_radius = 10
robot_angle = 0 # facing right
robot_speed = 2

# Maze walls matching your diagram — no extra rectangles at the bottom
obstacles = [
    # Outer walls (with entrance gap on left and exit gap on top)
    pygame.Rect(20, 125, 260, 10), # Top wall left part
    pygame.Rect(320, 125, 60, 10), # Top wall right part
    pygame.Rect(0, 125, 10, 150), # Left wall upper part
    pygame.Rect(0, 375, 10, 150), # Left wall lower part
    pygame.Rect(380, 125, 10, 400), # Right wall
    pygame.Rect(0, 505, 380, 10), # Bottom wall

    # Internal walls matching diagram
    pygame.Rect(50, 175, 200, 10), # Top horizontal

```

```

pygame.Rect(150, 175, 10, 50),      # Vertical drop from top

pygame.Rect(50, 225, 50, 10),      # Left-side horizontal
pygame.Rect(50, 225, 10, 50),      # Left-side vertical
pygame.Rect(50, 275, 50, 10),      # Left-side horizontal back

pygame.Rect(100, 275, 150, 10),     # Middle horizontal
pygame.Rect(250, 225, 10, 60),     # Middle vertical up

pygame.Rect(300, 175, 10, 100),     # Right-side vertical to exit
]

```

```

clock = pygame.time.Clock()

```

```

def sense_obstacle():
    """Check if robot is near any obstacle."""
    check_x = robot_pos[0] + math.cos(math.radians(robot_angle)) * 15
    check_y = robot_pos[1] + math.sin(math.radians(robot_angle)) * 15
    for obs in obstacles:
        if obs.collidepoint(check_x, check_y):
            return "near"
    return "far"

```

```

def decide(distance):
    """Decide what action to take based on distance sensor."""
    if distance == "near":
        action = random.choice(["turn_left", "turn_right", "stop"])
        reasoning = f"Obstacle detected → {action.replace('_', ' ').capitalize()}"
    else:
        action = "move_forward"
        reasoning = "No obstacle ahead → Moving forward"
    return action, reasoning

```

```

running = True
while running:
    screen.fill(WHITE)

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Simulate sensor
    distance = sense_obstacle()

```

```

# Decide action
action, reasoning = decide(distance)

# Perform action
if action == "move_forward":
    robot_pos[0] += robot_speed * math.cos(math.radians(robot_angle))
    robot_pos[1] += robot_speed * math.sin(math.radians(robot_angle))
elif action == "turn_left":
    robot_angle = (robot_angle - 90) % 360
elif action == "turn_right":
    robot_angle = (robot_angle + 90) % 360
elif action == "stop":
    pass # no movement

#
Draw maze
for obs in obstacles:
    pygame.draw.rect(screen, BLACK, obs)

# Draw robot
pygame.draw.circle(screen, BLUE, (int(robot_pos[0]), int(robot_pos[1])), robot_radius)
# Draw sensor line
end_x = robot_pos[0] + math.cos(math.radians(robot_angle)) * 20
end_y = robot_pos[1] + math.sin(math.radians(robot_angle)) * 20
pygame.draw.line(screen, GREEN, robot_pos, (end_x, end_y), 2)

# Draw text overlay on the right
text_x = 420
text_y = 50
draw_text(screen, f"Distance Sensor: {distance}", (text_x, text_y))
draw_text(screen, f"Action: {action}", (text_x, text_y + 25))
draw_text(screen, f"Reasoning:", (text_x, text_y + 50))
draw_text(screen, reasoning, (text_x, text_y + 75))

pygame.display.flip()
clock.tick(10)

pygame.quit()

```